RobotRover

System design, build notes and installation guide

Arduino UNO Q - Bluetooth LE - Android - Web Bluetooth - LED matrix

What this document covers

1. What the system is and the parts it is made of
2. How a command travels from a tap on the phone to a servo and the LED matrix
3. The seams: every place two components meet, what crosses, and how it fails
4. How the system was built, including the problems found along the way
5. How to install each part from scratch, and how to check it works
6. What has been tested, what has not, security, troubleshooting and a file map

Version 1.0 - October 21, 2026

Everything described here is open source and needs no accounts.

SECTION 1

At a glance

RobotRover is a small two-wheeled robot with a four-joint arm, driven from an Android phone or a desktop browser over Bluetooth. The brain is an **Arduino UNO Q**, a board with two processors: a Qualcomm chip running Debian Linux, and an STM32 microcontroller running the Arduino sketch. The board's 13 x 8 LED matrix shows what the robot is doing: an icon for each command, then a little face that reacts.

The design goal was a system that is **fully open source, needs no accounts, and works without an internet connection** once installed. Every tool used is either open source or downloaded directly from its publisher with checksum verification, and no component phones home.

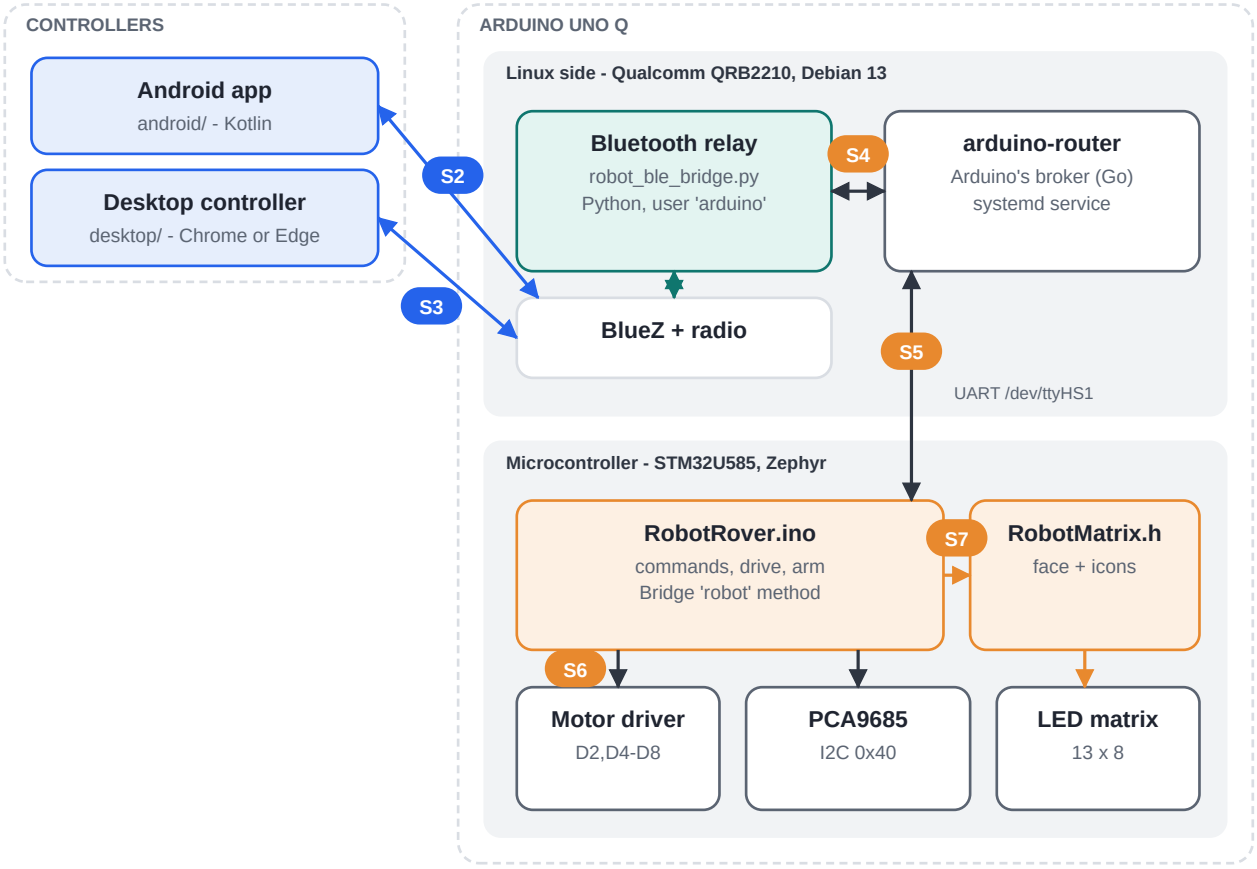


Figure 1. The components and the seams between them (S1, the command language, is shared by every seam and is not drawn). S8 and S9 are build-time seams and are covered in Section 4.

Why there are so many pieces

The UNO Q's Bluetooth radio is wired to the Linux processor, not to the microcontroller that drives the motors. So a phone command has to land on the Linux side first and then be handed across to the microcontroller. Arduino provides that hand-off: a broker called `arduino-router` on Linux and a library called **Bridge** in the sketch, connected by an internal serial link. RobotRover adds a small Python program, the **relay**, that speaks Bluetooth to the phone on one side and the router's protocol on the other.

Component	Runs on	Language	Job
Android app	Phone (Android 8+)	Kotlin, no libraries	Drive pad, arm sliders, shows replies
Desktop controller	PC browser (Chrome/Edge)	HTML + JavaScript	Same controls, keyboard, console, demo mode, matrix preview
Bluetooth relay	UNO Q, Linux side	Python 3.13	Bluetooth LE server; checks commands; forwards to the sketch
arduino-router	UNO Q, Linux side	Go (Arduino's)	Message broker between Linux programs and the microcontroller
RobotRover sketch	UNO Q, microcontroller	C++ (Arduino, Zephyr)	Runs commands: wheels, servos, safety timeouts
RobotMatrix	UNO Q, microcontroller	C++ header	Draws icons and the face on the LED matrix

The phone app

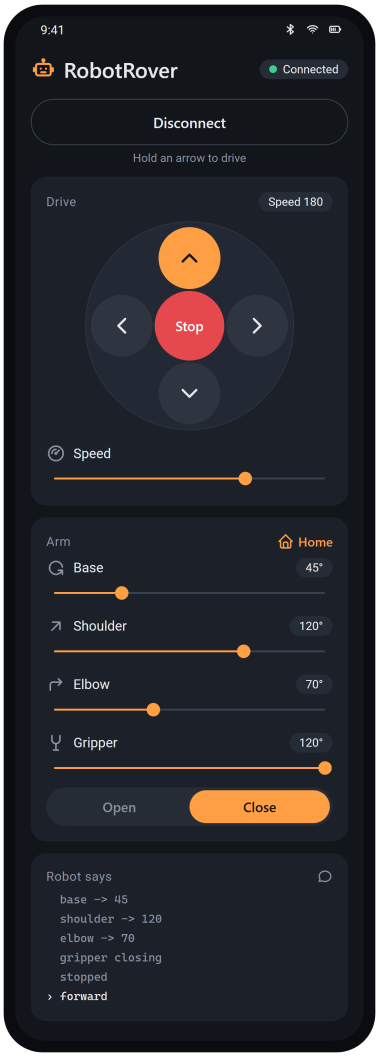


Figure 2 shows the Android app connected to the robot while the Forward arrow is held. From top to bottom:

- **Status pill:** grey when offline, yellow while searching or connecting, green when connected.
- **Connect / Disconnect** button, with a status line under it that explains problems (for example "Robot not found").
- **Drive card:** hold an arrow to drive; the held arrow lights orange. The red Stop sits in the middle. The pill and slider set the wheel speed.
- **Arm card:** one slider per joint with its angle in a pill. Home is in the corner. The Open / Close toggle follows the gripper slider.
- **Robot says:** the last six replies from the sketch, the newest one highlighted.
- Controls are dimmed and disabled until the app is connected.

The desktop controller (desktop/index.html) has the same controls in a wider layout, plus keyboard driving, a command console, a demo mode and a live preview of the LED matrix.

Figure 2. The Android app, connected, with Forward held. Mockup drawn from the app's own layout, colors and icons; on a phone the screen scrolls.

SECTION 2

The hardware

Arduino UNO Q

- **Linux side:** Qualcomm QRB2210, Debian GNU/Linux 13 (trixie), Python 3.13.5, hostname `uno-q`, default user `arduino`. Owns the Wi-Fi and Bluetooth 5.1 radio and the USB-C port.
- **Microcontroller:** STM32U585 (Cortex-M33), running Arduino's Zephyr-based core (board package *Arduino Q Boards* 1.0.0, FQBN `arduino:zephyr:unoq`). Owns the UNO-style header pins and the LED matrix.
- **Link between them:** an internal UART, opened on Linux as `/dev/ttyHS1` by arduino-router.
- **I/O voltage:** the header pins are 3.3 V. The TB6612 / L298N motor drivers and the PCA9685 servo board accept 3.3 V logic.

Robot parts

Part	Connection	Notes
Left wheel motor	PWM D5, direction D7 / D8	Through a dual H-bridge (TB6612FNG recommended, L298N works)
Right wheel motor	PWM D6, direction D4 / D2	Flip LEFT_REVERSED / RIGHT_REVERSED in the sketch if a wheel runs backwards
Front ball caster	-	Passive; follows the wheels
Servo board (PCA9685)	I2C SDA / SCL, address 0x40	16-channel, 50 Hz. Gives the servos their own power
Base servo	PCA9685 channel 0	0 - 180 deg, home 90
Shoulder servo	PCA9685 channel 1	15 - 165 deg, home 90
Elbow servo	PCA9685 channel 2	15 - 165 deg, home 90
Gripper servo	PCA9685 channel 3	30 (open) - 120 (closed) deg, home 30
LED matrix	Built into the UNO Q	13 x 8, 3-bit brightness (0 - 7)

Power. Motors and servos run from their own batteries, never from the UNO Q's 5 V pin. All grounds (board, motor driver, servo board, batteries) must be connected together.

Where everything connects

The pins RobotRover uses, colored by what they drive. The board is drawn to scale (68.6 x 53.3 mm) from Arduino's published top view of the UNO Q, with the header pins in their real positions.

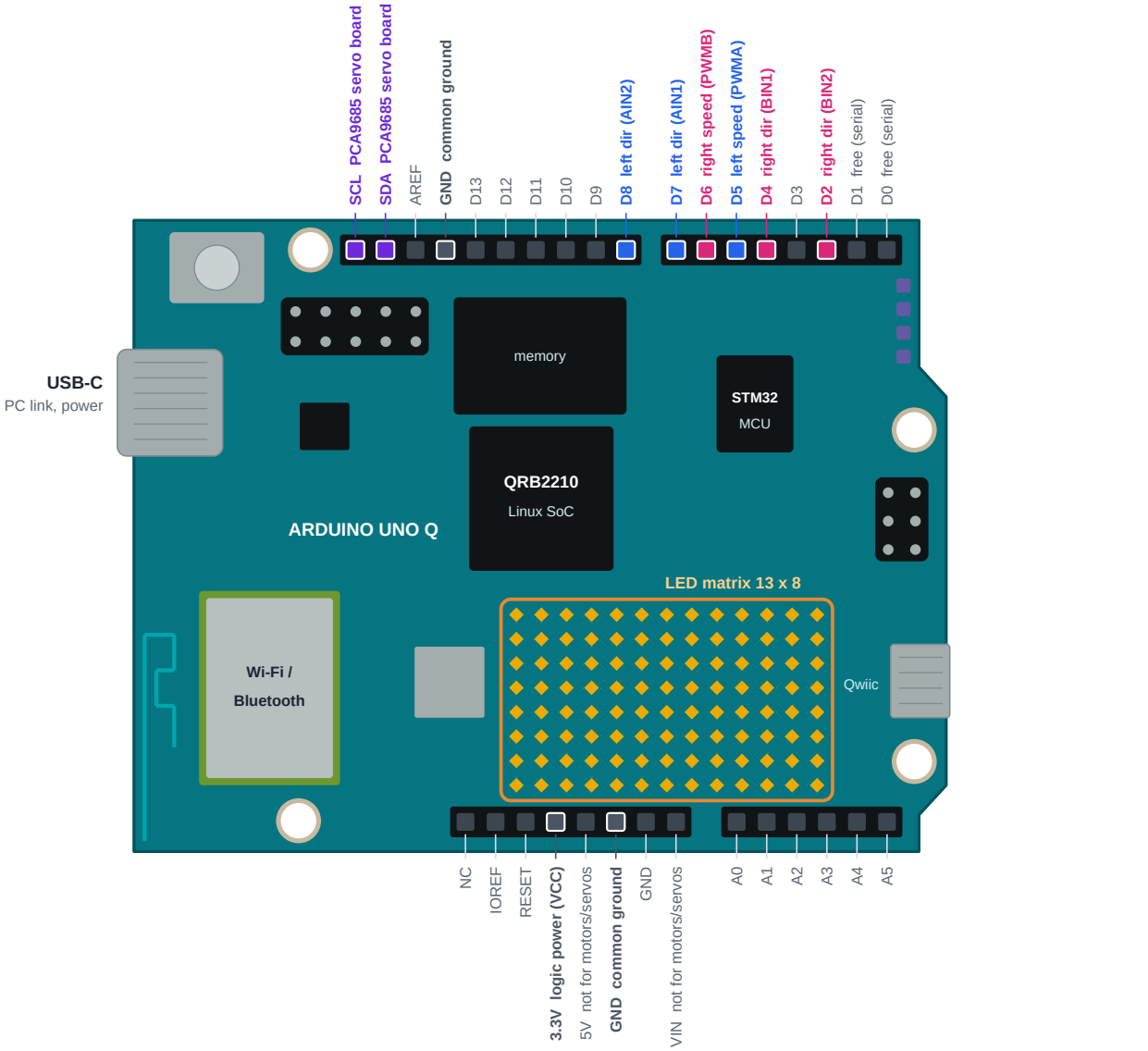


Figure 3. Arduino UNO Q pins used by RobotRover. Outline, headers, holes and major parts follow Arduino's top view; small parts are omitted and chip labels are approximate. Pins without a description are free. D5 and D6 must be PWM-capable pins.

Wiring

How the UNO Q, the TB6612FNG motor driver, the PCA9685 servo board, the motors, servos and batteries connect. Wire colors match Figure 3: blue left wheel, pink right wheel, purple I2C and servo cables, orange 3.3 V logic, red power, black ground.

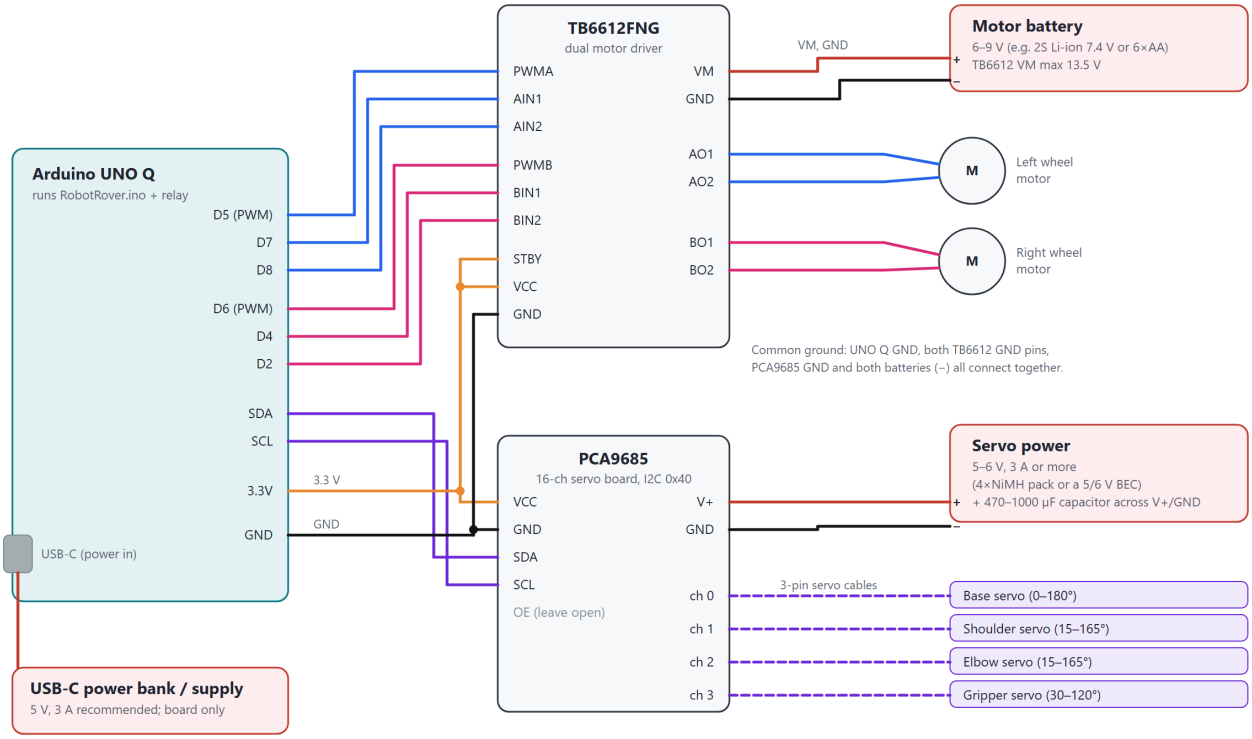


Figure 4. Wiring for RobotRover (schematic, not to scale). Pins match RobotRover.ino.

Rules that matter

- **Never power motors or servos from the UNO Q.** The board runs from a USB-C supply (Arduino specifies 5 V, 3 A); the motors and servos each get their own battery.
- **All grounds connect together:** UNO Q GND, both TB6612 GND pins, PCA9685 GND and both batteries' (-).
- **STBY must be tied high** (to 3.3 V), or the TB6612 ignores every command.
- **Logic is 3.3 V.** TB6612 VCC and PCA9685 VCC take 3.3 V from the board; the high-current power goes only to VM (motors, 6 - 9 V, TB6612 maximum 13.5 V) and V+ (servos, 5 - 6 V, 3 A or more).
- **Add a 470 - 1000 µF capacitor** across the servo board's V+ and GND to ride out current spikes.
- **Servo plugs:** signal (yellow or orange) toward the PCA9685's PWM row, brown or black toward GND.
- **Using an L298N instead?** ENA = D5, IN1 = D7, IN2 = D8, ENB = D6, IN3 = D4, IN4 = D2. Remove the ENA / ENB jumpers; motors go on OUT1 / OUT2 and OUT3 / OUT4.
- **First power-up:** wheels off the ground, then try `w 500` and `base 90` from the Serial Monitor or the desktop console.

SECTION 3

How a command travels

Following one command end to end is the quickest way to see how the parts fit. Here the user drags the Base slider on the phone to 45 degrees.

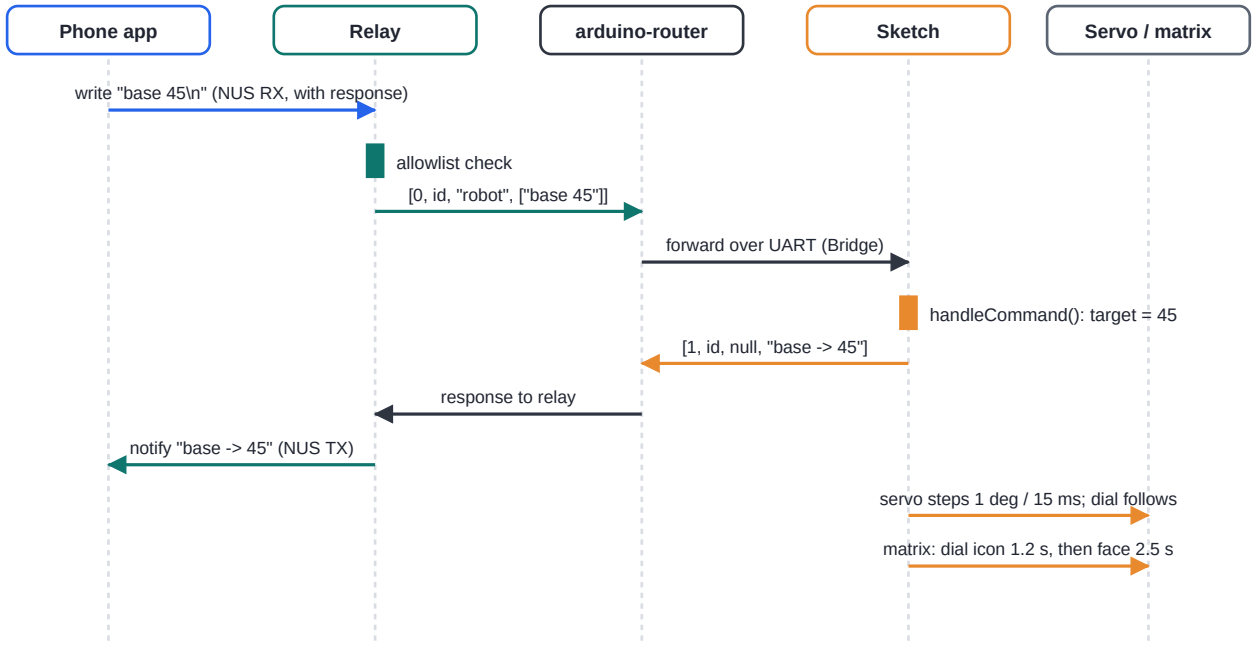
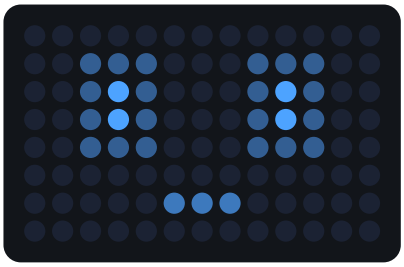


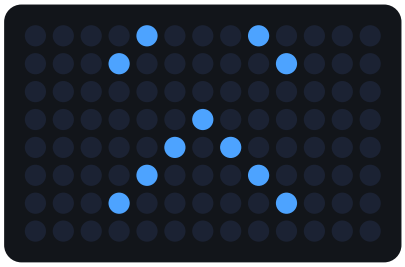
Figure 5. The life of "base 45". Each arrow crosses one seam from Section 4.

1. **Phone:** the slider sends `base 45` (at most every 100 ms while dragging, plus once on release). The app queues it, merging it with any older unsent `base` command, and writes it to the robot's Bluetooth characteristic.
2. **Relay:** receives the text, lower-cases it, checks it against an allowlist of robot commands, and calls the sketch's `robot` method through arduino-router's Unix socket.
3. **Router:** forwards the call over the internal UART to the microcontroller.
4. **Sketch:** Bridge runs the `robot` handler between two passes of `loop()`. It parses the command, clamps 45 to the base's limits, sets the target angle, tells the LED matrix, and returns the reply `base -> 45`.
5. **Back out:** the reply returns through the router and relay and arrives at the phone as a Bluetooth notification, shown under "Robot says".
6. **Physically:** the servo moves toward 45 degrees one degree every 15 ms. The LED matrix shows a "B" dial whose needle follows the servo's actual position, then the face glances toward the side the base turned.

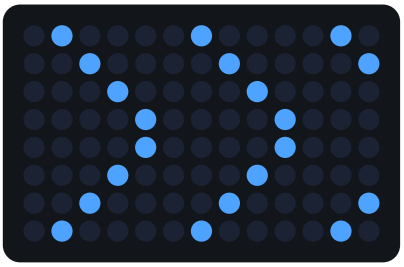
LED matrix frames



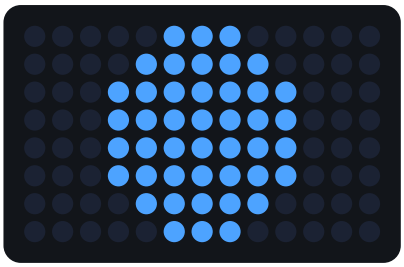
Idle face
Nothing happening; blinks every 4 s



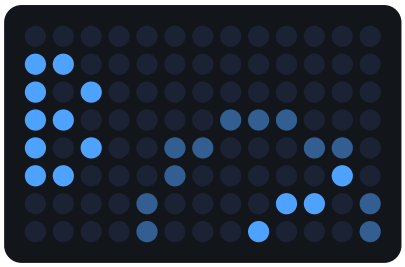
Forward
w held; the arrows stream upward



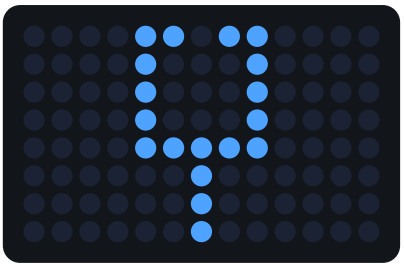
Spin right
d held; the arrows stream right



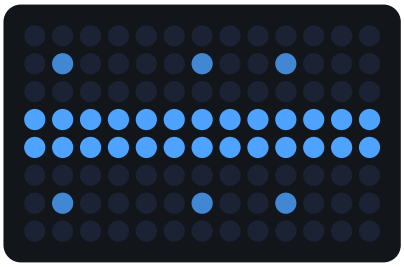
Stop
x; the sign pulses for 0.7 s



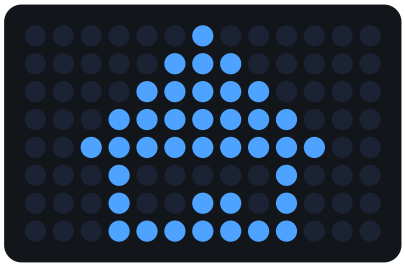
Base dial at 30 deg
The needle follows the real servo



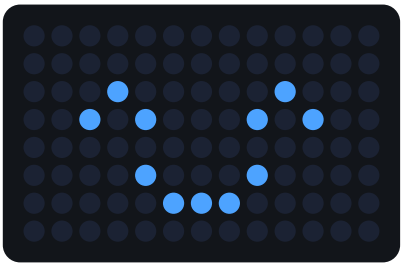
Gripper closed
The claw follows the real servo



Speed 255
Bar length is the speed; dots stream



Home
home; shown for 1.2 s



Happy face
For 2.5 s after home

Figure 6. Real frames from RobotMatrix.h, drawn from the same pixel data the board shows. Each command shows its icon first; the face reacts afterwards.

Driving is a stream of short pulses

Holding a drive arrow does not send "drive forever". The app sends `w 400` (forward for 400 ms) every 150 ms. Each pulse restarts the sketch's 400 ms timer. Releasing the button sends `x` (stop). If anything between the phone and the motors fails (Bluetooth drops, the app is backgrounded, the relay restarts), the pulses stop arriving and the wheels stop on their own within 400 ms. The same rule is used by the desktop controller.

SECTION 4

The seams

A seam is a place where two independently written components meet. Most bugs in a system like this live at the seams, so each one is described the same way: what crosses it, its exact format, which code owns each side, how it fails, and how to test it in isolation.

Seam	Between	Medium	Format
S1	Everything	-	The text command language
S2	Android app - relay	Bluetooth LE	Nordic UART Service, newline-terminated text
S3	Desktop controller - relay	Bluetooth LE (Web Bluetooth)	Same as S2
S4	Relay - arduino-router	Unix socket	MessagePack-RPC
S5	arduino-router - sketch	Internal UART	MessagePack-RPC via Bridge
S6	Sketch - motors and servos	GPIO / PWM / I2C	Pin levels, PCA9685 registers
S7	Sketch - LED matrix	Function calls	13 x 8 brightness frame
S8	C++ - JavaScript mirrors	Source code	Ported logic that must stay identical
S9	PC - devices (build and deploy)	USB	APK sideload, firmware upload, adb push

S1 The command language

Every controller, the relay and the sketch share one tiny text protocol. It is the contract that lets each side be written, tested and replaced independently: a command typed into the Serial Monitor, the desktop console, the relay's `--send` option, or sent from the phone all behave the same.

Command	Effect	Reply (examples)
w | s | a | d [ms]	Forward / back / spin left / spin right. Default 1000 ms; 0 = until stopped	forward, backward, spin left, spin right
x or stop	Stop the wheels	stopped
speed N	Wheel speed 0 - 255 (default 180)	speed 200
base | shoulder | elbow | grip N	Move a joint to N degrees, clamped to its limits	base -> 45, base -> 180 (limit 0-180)
open | close | home	Gripper open / closed, whole arm to home	gripper opening, arm going home
pos	Report targets and speed	base 90 shoulder 90 elbow 90 gripper 30 speed 180
help (Serial Monitor only)	Print the command list to the Serial Monitor	help printed to Serial Monitor

Rules: one command per line, lower case, single spaces, whole-number values. Replies are short single lines (they must fit in one Bluetooth packet; the relay truncates at 180 bytes). The relay only forwards commands matching this allowlist (from `board/robot_ble_bridge.py`); everything else is answered with `rejected: ...` and never reaches the sketch:

```
^(?:(?:w|a|s|d) \d{1,5}|w|a|s|d|x|stop|open|close|home|pos
  |(?:speed|base|shoulder|elbow|grip) \d{1,3})$
```

S2 Android app - relay (Bluetooth LE)

Crosses	Command text from the phone; reply text back.
Format	Nordic UART Service (NUS). Service 6e400001-b5a3-f393-e0a9-e50e24dcca9e. The phone writes to RX ...0002 (write with response), the relay notifies on TX ...0003. Each write is one command plus "\n"; the relay also splits a write on newlines.
Phone side	RobotBle.kt: scans for the service UUID or the name "RobotRover" (10 s timeout), connects with TRANSPORT_LE, requests a 185-byte MTU, discovers services, enables notifications by writing the 0x2902 descriptor, then sends through a one-at-a-time queue (max 20) that merges commands of the same kind (all drive commands are one kind).
Board side	robot_ble_bridge.py, built on bless 0.3.0 over BlueZ D-Bus. Advertises as "RobotRover". Commands go into an asyncio queue (max 50, oldest dropped) and are handled one at a time.
Permissions	Android 12+: BLUETOOTH_SCAN (neverForLocation) and BLUETOOTH_CONNECT. Android 8 - 11: location permission, and Location must be on. The app has no internet permission.
Fails how	"Robot not found": relay not running or Bluetooth off on the board. Connect then drop: radio range or the relay restarted. No replies but commands work: notifications not enabled.
Test it	Board: bluetoothctl show should list "Nordic UART Service" and ActiveInstances 1. Phone: the status pill turns green and "pos" (desktop console) gets a reply.

S3 Desktop controller - relay (Web Bluetooth)

Crosses	Exactly what S2 carries; the board cannot tell the PC and the phone apart.
Format	Same NUS service and characteristics. Uses navigator.bluetooth.requestDevice with a filter on the service UUID or name, writeValueWithResponse, and startNotifications.
PC side	desktop/index.html, served by desktop/serve.py on http://localhost:8765 bound to 127.0.0.1 (Web Bluetooth needs a secure origin; localhost counts). Chrome or Edge only.
Extras	Keyboard driving, a console for raw commands, demo mode (no hardware), and an LED matrix preview. When connected, the preview mirrors only commands sent from this page.
Fails how	"This browser can't use Bluetooth": wrong browser or the file was opened directly instead of through serve.py. Device list empty: PC Bluetooth off or relay not advertising.
Test it	Try demo mode first (exercises everything except Bluetooth), then Connect.

S4 Relay - arduino-router (MessagePack-RPC over a Unix socket)

Crosses	One RPC call per command: method "robot", one string parameter; the reply string back.
Format	MessagePack arrays on /var/run/arduino-router.sock (mode 666, so no root needed). Request [0, msgid, "robot", ["base 45"]]; response [1, msgid, error, result].
Relay side	RouterClient in robot_ble_bridge.py: a minimal client (about 60 lines) instead of Arduino's App Lab Python package, so the relay runs outside App Lab. 3 s timeout, one reconnect attempt on a dropped socket, ignores unrelated messages.
Router side	arduino-router (GPL-3.0) brokers calls in a star topology. Methods are registered by whoever provides them; here the sketch registers "robot".
Fails how	[2, 'method robot not available']: the sketch is not running or is an old version without Bridge.provide_safe("robot"). Timeout: the sketch is stuck or rebooting. Connection refused: arduino-router is not running.
Test it	./run.sh --send pos on the board (no Bluetooth involved). Expected: base 90 shoulder 90 elbow 90 gripper 30 speed 180.

S5 arduino-router - sketch (Bridge over the internal UART)

Crosses	The forwarded RPC request and the sketch's reply.
Format	MessagePack-RPC over UART /dev/ttyHS1. Handled entirely by Arduino's Bridge library (Arduino_RouterBridge) in the sketch; RobotRover never touches the bytes.
Sketch side	setup() calls Bridge.begin(), Monitor.begin(), then Bridge.provide_safe("robot", bridgeCommand). "Safe" handlers run in the main thread between loop() passes (the core calls __loopHook after each loop), so they never race with the motor and servo code. loop() must therefore stay fast and non-blocking.
Fails how	A burst of "invalid packet, expected array" in the router log right after the microcontroller resets is normal (Bridge flushes old data at startup). If the sketch blocks in loop(), every command times out at S4.
Test it	journalctl -u arduino-router shows the connection; S4's --send test proves S5 too.

S6 Sketch - motors and servos

Crosses	Electrical signals.
Wheels	Each motor: one PWM pin (analogWrite 0 - 255) and two direction pins. IN1 high = forward, IN2 high = reverse, both low = coast. A timed drive stops itself when driveStopAt passes.
Servos	Adafruit PCA9685 library over I2C (address 0x40), 50 Hz, angle mapped to 500 - 2500 us. updateArm() moves each servo 1 degree every 15 ms toward its target, so motion is smooth and the power draw is spread out.
Fails how	"PCA9685 servo board not found" on the Serial Monitor at startup: wiring or power. Servos jitter or the board resets: servos are powered from the board, or grounds are not shared. Note: servos jump to home at power-on because they cannot report position.
Test it	Serial Monitor commands (base 45, w 500) with the robot on a stand.

S7 Sketch - LED matrix

Crosses	Two calls: RobotMatrix::onCommand(cmd, hasValue, value) from handleCommand, and RobotMatrix::update(angles, speed) from every loop().
Format	A 104-byte frame (13 columns x 8 rows, row-major), values 0 - 7, passed to Arduino_LED_Matrix::draw() after setGrayscaleBits(3). At most one frame every 40 ms.
Behavior	An icon for 0.6 - 1.5 s (refreshed while a command repeats), then a face reaction for 2.5 s, then the idle face (blinks every 4 s, glances every 9 s). The joint dial and claw draw the servos' current angles, not the targets.
Fails how	Matrix blank: sketch not running. All LEDs the same brightness or very dim: the grayscale setting; online reports about the UNO Q's brightness range conflict, so check this on the real board first.
Test it	Power the bare board: the idle face should appear within a second.

S8 Mirrors: C++ logic ported to JavaScript

The desktop controller's demo mode and its matrix preview contain line-for-line ports of sketch code. This is a source-code seam: nothing checks it automatically, so any change on one side must be copied to the other.

C++ original	JavaScript copy (desktop/index.html)	How it was verified
handleCommand() in RobotRover.ino	demoSketch()	Same replies for the full command table, including clamping messages
Relay allowlist (Python regex)	ALLOWED	Same pattern; demo rejects "help" exactly like the real relay
updateArm(): 1 degree / 15 ms	armClock interval	Dial and claw animate at the same speed
RobotMatrix.h (all drawing)	LM module	C++ compiled on the PC with g++ and stub libraries; its frames matched the JavaScript frames pixel for pixel for 10 test cases

S9 Build and deploy seams

Path	Tool	What crosses	Notes
PC -> phone	MTP file transfer	RobotRover.apk	Sideloaded from the Download folder. Debug-signed; Play Protect may warn.
PC -> microcontroller	arduino-cli (inside Arduino IDE 2) on COM13	Compiled sketch	Upload goes through the board's Linux side, which flashes the STM32 over SWD with OpenOCD.
PC -> board Linux	adb (platform-tools 37.0.1) over USB	Relay files and libraries	The board exposes an ADB interface on USB; no password needed for the arduino user.
Board boot	cron @reboot -> keepalive.sh	Starts the relay	Restarts it 3 s after any exit.

Values that must match across seams

If you change any of these, change every place listed.

Value	Where it lives
Service UUID 6e400001-..., RX ...0002, TX ...0003	robot_ble_bridge.py, RobotBle.kt, desktop/index.html
Device name "RobotRover"	robot_ble_bridge.py, RobotBle.kt, desktop/index.html
Bridge method name "robot"	RobotRover.ino (provide_safe), robot_ble_bridge.py (BRIDGE_METHOD)
Command allowlist	robot_ble_bridge.py (ALLOWED), desktop/index.html (ALLOWED)
Joint limits and home angles	RobotRover.ino (joints), activity_main.xml (slider min/max), MainActivity.kt (HOME_*), desktop/index.html (sliders, demo, HOME)
Gripper open 30 / closed 120	RobotRover.ino, RobotMatrix.h (claw), MainActivity.kt, desktop/index.html
Drive pulse 400 ms every 150 ms	MainActivity.kt, desktop/index.html
Reply length under 180 bytes	robot_ble_bridge.py (MAX_REPLY_BYTES), app MTU request (185)
Matrix shapes and timings	RobotMatrix.h and the LM module in desktop/index.html

SECTION 5

How it was built

The project was built in one long session, in this order. The decisions and the problems found along the way are recorded here because they explain why things are the way they are.

Decisions

Decision	Why
Arduino IDE 2 / arduino-cli instead of App Lab	Open source, works offline once the board package is installed, no account.
Relay on the Linux side	The Bluetooth radio is only reachable from Linux; Arduino's Bridge is the supported way across.
Bluetooth LE with the Nordic UART Service	Widely supported text-over-BLE convention; easy to test with generic tools.
Own MessagePack-RPC client in the relay	Avoids depending on App Lab's Python package; about 60 lines.
Command allowlist in the relay	BLE here has no pairing, so the relay limits what a stranger could send.
Drive pulses instead of start/stop	Fails safe: losing the link stops the wheels within 400 ms.
PCA9685 for servos	Arduino's Servo library support on the UNO Q's Zephyr core could not be confirmed; the arm also needs separate servo power anyway.
Kotlin app with zero libraries	Small (838 KB), nothing to audit beyond the code, no internet permission.
Dark theme, round drive pad	Chosen from an interactive mockup before implementation.
Desktop controller via Web Bluetooth	Tests the same seam as the phone from a PC; demo mode needs no hardware.
"Face + icons" LED matrix design	Chosen from four animated concepts: clear about what is running, with personality.
SDK parts downloaded directly, not via Google's 'android' tool	The new tool collects usage data by default; direct downloads with checksums send nothing.
Relay installed offline as the normal user	The board had no internet and sudo needs a password; BlueZ's D-Bus policy and the router socket both allow a normal user.

Problems found and fixed

Problem	Cause	Fix
Phone could not connect	Relay never installed; board had no internet for setup.sh	Packages bundled on the PC, pushed with adb, run as user 'arduino', started by cron
Relay got "method robot not available"	Board still had the first sketch version, from before the Bluetooth work	Compiled and uploaded the current sketch with arduino-cli
pip downloaded the wrong packages	Dependency markers were evaluated for Windows, not the board	Downloaded exactly five pinned packages with --no-deps

Problem	Cause	Fix
sdkmanager no longer manages licenses	Google replaced it with the 'android' CLI, which collects usage data	Read the license, user accepted it, downloaded packages directly
Keyboard driving did not start in a test	Handler assumed every key event came from an element	Guarded the check; real keypresses and synthetic ones now both work
Matrix preview stayed blank in the in-app browser	requestAnimationFrame does not fire there	Switched to a 20 ms timer (also keeps running when the window is covered)
Relay start command killed itself	pkill -f matched its own command line	Used the pattern [r]obot_ble_bridge.py

Tool and package versions

Area	Versions
Microcontroller	Arduino Q Boards (arduino:zephyr) 1.0.0, arduino-cli 1.5.1, Arduino_RouterBridge, Adafruit PWM Servo Driver Library, Arduino_LED_Matrix (bundled)
Board Linux	Debian 13, Python 3.13.5, BlueZ; relay packages bless 0.3.0, bleak 3.0.2, dbus-fast 5.2.0, dbus-next 0.2.3, msgpack 1.2.3
Android	Gradle 8.9, Android Gradle Plugin 8.7.3, Kotlin 2.0.21, compileSdk / targetSdk 35, minSdk 26, build-tools 34.0.0, JDK 21
Tools on the PC	Android SDK in D:\Android\Sdk (platform 35 r02, build-tools 34.0.0, platform-tools 37.0.1), Python 3.12 for the desktop server

SECTION 6

Installation

Install in this order: the sketch, then the relay, then a controller. Each step ends with a check that proves its seam works before moving on.

Prerequisites

Needed for	What
Sketch	Arduino IDE 2 (or arduino-cli) with the Arduino Q Boards package and the two libraries
Relay	USB cable and adb (Android platform-tools), or SSH and internet on the board
Android app	JDK 17 or newer, the Android SDK parts below (or Android Studio), a USB cable
Desktop controller	Python 3 and Chrome or Edge, on a PC with Bluetooth

A. The sketch (microcontroller)

1. In Arduino IDE 2: **Boards Manager** - install **Arduino Q Boards**. **Library Manager** - install **Arduino_RouterBridge** and **Adafruit PWM Servo Driver Library**.
2. Open RobotRover.ino (RobotMatrix.h must sit in the same folder), select **Arduino UNO Q**, and upload. Or from a terminal in the project folder:

```
arduino-cli compile -b arduino:zephyr:unoq .
arduino-cli upload  -b arduino:zephyr:unoq -p COM13 .     # use your board's port
```

Check: the LED matrix shows the idle face. In the Serial Monitor (line ending "New Line"), `pos` answers with the joint angles.

B. The Bluetooth relay (board Linux side)

Option 1: offline over USB, no password (how the current board is set up)

1. Install Android platform-tools to get adb, and check the board is listed with `adb devices`.
2. On the PC, download the five packages built for the board (aarch64, Python 3.13) and unpack them into a `lib` folder next to the relay files:

```
python -m pip download --no-deps --only-binary=:all: --dest wheels \
    --platform manylinux2014_aarch64 --platform manylinux_2_28_aarch64 \
    --python-version 3.13 --implementation cp --abi cp313 --abi none \
    bless==0.3.0 bleak==3.0.2 dbus-fast==5.2.0 dbus-next==0.2.3 msgpack==1.2.3
# unzip every .whl in wheels/ into robot-ble/lib/  (a wheel is a zip file)
```

3. Copy the libraries and the three relay files to `/home/arduino/robot-ble/` (from the project folder), then make the scripts executable and start the relay at boot:

```
adb shell "mkdir -p ~/robot-ble"
adb push robot-ble/lib /home/arduino/robot-ble/
adb push board/robot_ble_bridge.py board/run.sh board/keepalive.sh /home/arduino/robot-ble/
adb shell "chmod +x ~/robot-ble/*.sh"
# note: the next line replaces the arduino user's crontab (it is empty on a fresh board)
adb shell "echo '@reboot /home/arduino/robot-ble/keepalive.sh' | crontab -"
adb shell "cd ~/robot-ble && setsid nohup ./keepalive.sh >/dev/null 2>&1 </dev/null &"
```

Check S4/S5: `adb shell "cd ~/robot-ble && ./run.sh --send pos"` prints the joint angles. **Check S2:** `adb shell "bluetoothctl show"` lists the Nordic UART Service, and `~/robot-ble/relay.log` says "Advertising as 'RobotRover'".

Option 2: standard install (board has internet)

Copy the `board` folder to the board (scp or adb push) and run `bash setup.sh`. It installs packages with apt and pip, puts the relay in `/opt/robot-ble` and runs it as a root systemd service (`robot-ble-bridge`). It asks for the sudo password.

C. The Android app

1. Get the SDK parts: either install Android Studio, or download **platform 35** and **build-tools 34.0.0** from Google's repository (the Android SDK License Agreement applies) into a folder such as D:\Android\Sdk, and point `android/local.properties` at it (`sdk.dir=D\:\\Android\\Sdk`).
2. Build with a JDK 17+ as JAVA_HOME:

```
cd android
gradlew.bat assembleDebug          # output: app/build/outputs/apk/debug/app-debug.apk
```

3. Plug the phone in with **File transfer** on and copy the APK to its **Download** folder.
4. On the phone: **Files - Downloads**, tap the APK, allow **Install unknown apps** for Files, tap **Install**, and choose **Install anyway** if Play Protect warns.

Check: open RobotRover, tap **Connect to robot**, allow the Bluetooth permission. The pill turns green; Home or Open/Close produce replies and matrix icons.

D. The desktop controller

```
python desktop/serve.py            # opens http://localhost:8765 in your browser
```

Use Chrome or Edge. **Try demo** works without hardware; **Connect to robot** opens the browser's Bluetooth device picker. Keyboard: W/A/S/D or arrows to drive, Space stop, O/C/H gripper and home.

Verification checklist

#	Check	Proves
1	Matrix shows the idle face after power-on	Sketch running (S7)
2	Serial Monitor: pos answers	Sketch command handling (S1)
3	./run.sh --send pos answers on the board	Relay - router - sketch (S4, S5)
4	bluetoothctl show lists the Nordic UART Service	Relay advertising (S2)
5	Phone connects and Home gets "arm going home"	Whole chain (S2 to S7)
6	Hold an arrow, then turn the phone's Bluetooth off: wheels stop within 0.4 s	Drive safety
7	Reboot the board; phone reconnects after about a minute	Boot autostart (S9)

SECTION 7

What has been tested

Verified	How
Phone app connects to the board over Bluetooth	On the real phone (moto g stylus 5G 2023) and board
Relay - router - sketch round trip	./run.sh --send pos and home on the board
Relay advertising and autostart setup	bluetoothctl show; keep-alive running; crontab entry present
Sketch compiles for the UNO Q and uploads	arduino-cli 1.5.1, Arduino Q Boards 1.0.0
Android app builds cleanly	Gradle 8.9 / AGP 8.7.3, no compiler warnings; APK permissions inspected with aapt2
RobotMatrix.h drawing logic	Compiled on the PC with g++ -Wall -Wextra and stubs; 10 frames checked
C++ and JavaScript matrix match	Pixel-identical frames for the same inputs
Desktop demo mode, keyboard and console	Automated checks in a browser
Relay protocol code and allowlist	Unit test against a fake router, including the error path

Not yet verified	Notes
Motors, servos and the PCA9685 in the robot	Not wired yet; test on a stand first
LED matrix brightness levels on the real board	Conflicting reports online about the grayscale range
Desktop controller over real Bluetooth	Demo mode only so far
Wheels stop when the link drops	Logic is in place; check with item 6 of the checklist
Relay restart after a reboot	Set up but not yet rebooted; checklist item 7

SECTION 8

Security and privacy

- **No accounts anywhere.** No Arduino Cloud, no Google sign-in, no App Lab account.
- **Nothing phones home.** The Android app has no internet permission. The desktop page loads nothing from the internet and its server only listens on 127.0.0.1. Google's new 'android' SDK tool (which collects usage data by default) was not used for installs; its two help commands were run once before that was known.
- **Bluetooth is open.** There is no pairing or PIN, so anyone nearby with a Bluetooth app could connect while the relay runs. The relay's allowlist means they can only send robot commands, never anything to the Linux system. Stop the relay when not in use, or add a PIN in the relay as a future step.
- **Least privilege.** The relay runs as the normal 'arduino' user, not root.
- **Downloads were verified.** Every SDK and Gradle download was checked against the publisher's SHA-1 or SHA-256; Python packages came from pypi.org over HTTPS with pip's hash checks.
- **License.** Building the Android app uses Google's Android SDK, under the Android SDK License Agreement (accepted for this project). The RobotRover code itself does not depend on it at runtime.

SECTION 9

Troubleshooting by seam

Symptom	Seam	Check
App: "Robot not found"	S2	Relay running? tail ~/robot-ble/relay.log; bluetoothctl show (Powered: yes). Android 11 or older: Location on.
Connects, but replies say error: 'robot' failed ... not available	S4/S5	The sketch on the board is old or not running. Re-upload RobotRover.ino.
Replies say error: no answer from 'robot'	S5	Sketch stuck or rebooting; check the Serial Monitor.
Replies say rejected: ...	S1	The command is not in the allowlist (expected for help or typos).
Matrix blank	S7	Sketch not running; re-upload.
"PCA9685 servo board not found"	S6	SDA/SCL wiring, servo board power, shared ground.
A wheel runs backwards	S6	Flip LEFT_REVERSED or RIGHT_REVERSED.
Desktop: "This browser can't use Bluetooth"	S3	Use Chrome or Edge, opened through serve.py.
Relay not running after reboot	S9	crontab -l shows the @reboot line; relay.log shows "relay starting".

SECTION 10

File map

Path	What it is
RobotRover.ino	The sketch: commands, wheels, servos, Bridge registration
RobotMatrix.h	LED matrix icons and face (part of the sketch)
board/robot_ble_bridge.py	The Bluetooth relay, with its MessagePack-RPC client and allowlist
board/run.sh, board/keepalive.sh	Run the relay with bundled libraries; keep it running (offline install)
board/setup.sh, board/robot-ble-bridge.service	Standard install with apt/pip and a systemd service
android/	Android app (Kotlin). Gradle wrapper included; local.properties is per-PC and not in git
android/app/src/main/java/.../RobotBle.kt	Bluetooth LE client: scan, connect, queue, notifications
android/app/src/main/java/.../MainActivity.kt	Screen logic: drive pulses, sliders, status, log
android/app/src/main/res/	Layout, theme, colors, icons, launcher icon
desktop/index.html	Desktop controller, demo model and matrix preview (ports of sketch code)
desktop/serve.py	Local web server on 127.0.0.1:8765
README.md	Quick start and reference
RobotRover_System_Guide.pdf	This document
images/	App mockup, board pinout and wiring images (used in the README and this document)